BCA · ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

ATM AND UPI TRANSACTION SIMULATOR

A Python-based simulation of everyday banking operations

PROJECT REPORT

Submitted by

PROJECT TEAM

M. Harshith	SRN: R25DA086
M. Amrutha Varshini	SRN: R25DA085
Mariyum	SRN: R25DA087

Section: B | BCA · III Semester

BCA – III Semester
REVA University

Academic Project · 2026

DECLARATION

I hereby present this project report titled **“ATM and UPI Transaction Simulator”** as an academic project demonstrating the application of Python programming concepts. The implementation and project details documented here are based on the source code, presentation, and testing report supplied for this project.

PROJECT DETAILS

Project title	ATM and UPI Transaction Simulator
Project team	M. Harshith · R25DA086 M. Amrutha Varshini · R25DA085 Mariyum · R25DA087
Section	B
Programme	BCA · Artificial Intelligence and Machine Learning
Semester	III Semester
Institution	REVA University
Implementation language	Python
Execution environment stated in presentation	Python 3.13 (Python 3.10+ compatible); Jupyter Notebook / Python-compatible terminal

ACKNOWLEDGEMENT

I express my sincere gratitude to REVA University and the faculty members of the BCA programme for providing the academic environment and guidance needed to complete this project. I also acknowledge the support of my project team members, M. Amrutha Varshini and Mariyum, as listed in the project presentation.

This report brings together the submitted project presentation, Python implementation, and separate 15-case testing document into one structured project record.

ABSTRACT

The ATM and UPI Transaction Simulator is a menu-driven Python application created for learning and demonstration. It models common banking actions such as PIN-based access, balance enquiry, cash withdrawal, deposit, UPI transfer, mini-statement generation, and PIN change. The program applies validation rules, withdrawal and transfer limits, a minimum-balance requirement, cash denomination calculation, basic fraud-alert logic, and local PIN-file handling. Transactions are stored in memory as tuples within a list. The project demonstrates practical use of loops, conditions, functions, lists, tuples, slicing, file handling, and exception handling. It is a training simulation and does not connect to a real bank or payment network.

KEYWORDS

Python · ATM simulation · UPI transfer · PIN verification · transaction history · input validation · file handling

TABLE OF CONTENTS

1. Project Overview

2. Objectives

3. Scope and Features

4. Tools, Requirements and Python Concepts

5. System Design and Control Flow

6. Implementation Details

7. Testing Methodology and Test Cases

8. Results and Discussion

9. Limitations and Future Enhancements

10. Conclusion

Appendix A – Complete Python Source Code

Appendix B – Presentation Visuals

Note: The project presentation lists the team as M. Amrutha Varshini, Mariyum, and Harshith. This report is prepared with M. Harshith as the submitting student, using the SRN and section details supplied in the conversation.

1. PROJECT OVERVIEW

The **ATM and UPI Transaction Simulator** is a menu-driven Python application designed to simulate basic banking operations in a safe training environment. It allows a user to authenticate with a four-digit PIN and then select from seven menu options. The simulator applies simple banking rules to each operation and displays the resulting balance or validation message.

The application is not connected to a real bank, ATM network, or UPI payment system. All account values and transactions are simulated within the running program.

2. PROJECT OBJECTIVES

• Simulate common ATM and UPI banking operations.

• Implement PIN verification with a maximum of three login attempts.

• Apply transaction limits and minimum-balance restrictions.

• Calculate cash denominations using ■500, ■200, and ■100 notes.

• Maintain transaction records using a list containing tuples.

• Demonstrate menu-driven programming and Python control flow.

• Use local file handling to load and save the user's PIN.

3. SCOPE AND FEATURES

PIN verification	Checks a four-digit PIN and allows up to three attempts. PIN input is hidden using getpass.
Balance enquiry	Displays the current simulated account balance.
Cash withdrawal	Validates numeric input, positive amount, ■100 multiples, daily limit and minimum balance; then calculates notes.
Deposit	Adds a positive numeric amount to the simulated balance.
UPI transfer	Checks basic UPI ID formatting, positive amount, per-transaction limit and minimum balance.
Fraud alert	Displays an alert after three consecutive successful UPI transfers of ■3,000 or more.
Mini-statement	Displays up to the five latest transactions from the current session.
Change PIN	Checks the current PIN, validates and confirms a new four-digit PIN, then saves it in pin.txt.
Exit	Ends the menu loop and displays a thank-you message.

4. TOOLS, REQUIREMENTS AND PYTHON CONCEPTS

4.1 Software requirements

• Programming language: Python 3.13, as stated in the presentation (Python 3.10+ compatible).
• Execution environment: Jupyter Notebook or a Python-compatible terminal.
• Imported modules: os and getpass.
• Operating system: A system capable of running a compatible Python interpreter.

4.2 Hardware requirements

A basic computer or laptop capable of running Python is sufficient. The supplied project materials do not specify minimum processor, memory, or storage requirements.

4.3 Python concepts used

Concept	Use in the project
Variables and constants	Store PIN file name, balance, limits, counters and denominations.
Conditional statements	Validate choices, PINs, transaction amounts and banking rules.
for and while loops	Control PIN attempts, repeat the ATM menu and calculate note counts.
for-else and break	Show card-blocked output only when all three PIN attempts fail.
Functions	record_transaction() centralizes transaction recording.
Lists and tuples	Transactions are tuples stored in a list; cash-note counts are also stored as tuples.
Slicing	transactions[-5:] selects the five latest transaction records.
File handling	Read and write the PIN in the local pin.txt file.
Exception handling	try-except catches non-numeric transaction amount input.
Modules	os checks for the PIN file; getpass hides PIN entry in terminal.

5. SYSTEM DESIGN AND CONTROL FLOW

5.1 Startup and authentication

At startup, the program checks whether pin.txt exists. If it exists, the stored text is read and checked for a four-digit numeric format. If no usable saved PIN is available, the program uses the default PIN 1234. The user is then prompted up to three times. A successful match opens the menu; three failed attempts end the session with a card-blocked message.

5.2 Banking menu

After successful authentication, the program repeatedly displays the seven menu options. The user's choice is stripped of surrounding whitespace and validated. Invalid menu choices display an error and return to the menu.

5.3 Transaction processing

For successful deposits, withdrawals, and UPI transfers, the balance is updated and a transaction tuple is appended to the transactions list. The mini-statement reads only the latest five records. PIN changes are also recorded with an amount of zero.

5.4 Core transaction rules

Initial balance	■10,000.00
Minimum balance	■500.00
Daily cash withdrawal cap	■20,000.00 per program session
UPI transfer cap	■5,000.00 per transaction
Large-transfer threshold	■3,000.00 or more
Fraud alert trigger	Three consecutive successful qualifying UPI transfers
Cash denominations	■500, ■200 and ■100
PIN attempts	Maximum of three attempts

6. IMPLEMENTATION DETAILS

6.1 PIN file handling

The program uses os.path.exists() to check for pin.txt. The saved PIN is read using a with open(...) block and strip() removes surrounding whitespace. During a PIN change, the updated value is written back to the same file. The file stores the PIN as plain text.

6.2 Transaction recording

The function record_transaction(transaction_type, amount) creates a four-element tuple: transaction ID, transaction type, amount, and balance after the transaction. The tuple is appended to the transactions list. Transaction IDs are assigned using the current list length plus one.

6.3 Cash denomination algorithm

The withdrawal amount is processed against the denominations tuple in descending order: ■500, ■200, then ■100. A for loop selects each denomination, while a nested while loop counts how many notes can be used. The remaining amount is reduced after each note. The balance is changed only after the amount has passed the validation and dispensing checks.

6.4 UPI validation and fraud alert

The UPI ID check is a basic format check: it must contain an @ symbol, must not start or end with @, and must not contain spaces. A successful transfer reduces the balance and is recorded. Transfers of ■3,000 or more increase the consecutive-transfer counter. After the third qualifying transfer, a fraud alert is printed and the counter resets. Invalid transfers and non-UPI menu actions reset the counter.

6.5 Mini-statement and session behaviour

The mini-statement displays transactions[-5:], so no more than five recent records are shown. The transactions list, daily withdrawal counter, and fraud streak are initialized in memory when the program starts. The presentation explicitly identifies session-only transaction history and resettable daily withdrawal tracking as limitations.

6.6 PIN change

The user must provide the current PIN, enter a different four-digit numeric PIN, and confirm it. If the values match, the new PIN is saved to pin.txt. A PIN Change record is added to the current session’s transaction list.

6.7 Important implementation note

The application is an educational simulation. The PIN is stored locally in plain text, UPI ID validation is basic, and the program does not communicate with a bank or payment network. These limitations are stated in the supplied presentation.

7. TESTING METHODOLOGY AND TEST CASES

The separate testing document contains 15 test cases grouped under PIN verification, ATM operations, deposit and UPI operations, fraud alert and transaction history, and PIN management. All 15 cases are marked PASS in the supplied testing report.

7.1 Test case summary

TC01	Successful PIN verification	Correct saved PIN	Login successful and ATM menu displayed.	**PASS**
TC02	Incorrect PIN verification	One incorrect PIN, then correct PIN	Incorrect PIN message and remaining attempts, followed by successful login.	**PASS**
TC03	Card blocking after three incorrect attempts	Three incorrect PINs: 1111, 2222, 3333 (assuming none is correct)	Card-blocked message after three failed attempts.	**PASS**
TC04	Balance enquiry	Select option 1	Current balance displayed.	**PASS**
TC05	Successful cash withdrawal and denomination calculation	Select option 2; withdraw ■1,800	Withdrawal succeeds; 3×■500, 1×■200 and 1×■100 notes.	**PASS**

The test status shown above reproduces the PASS status in the supplied test report; it is not a new independent execution of the program.

TC06	Invalid cash withdrawal amount	Select option 2; enter ■50	Withdrawal rejected because it is not payable using the available denominations / does not meet ■100 multiple validation.	**PASS**
TC07	Minimum balance restriction	Starting balance ■10,000; withdraw ■9,600	Rejected because remaining balance would be below ■500.	**PASS**
TC08	Daily withdrawal limit	Fresh session; deposit ■50,000; withdraw ■10,000 twice; then try ■100	First two withdrawals succeed; next ■100 is rejected after ■20,000 daily limit.	**PASS**
TC09	Successful deposit	Select option 3; deposit ■20,000	Deposit succeeds and balance increases by ■20,000.	**PASS**
TC10	Successful UPI transfer	UPI ID person1@upi; transfer ■1,000	Transfer succeeds and balance decreases by ■1,000.	**PASS**

TC11	UPI transaction limit	UPI ID person1@upi; transfer ■6,000	Rejected because per-transaction UPI cap is ■5,000.	**PASS**
TC12	Fraud alert for consecutive large UPI transfers	a@upi ■3,000; b@upi ■3,500; c@upi ■4,000	Fraud alert after the third consecutive qualifying transfer.	**PASS**
TC13	Mini-statement generation	Perform six transactions, then select option 5	Only the latest five transactions are displayed with IDs, types, amounts and balances.	**PASS**
TC14	Empty mini-statement	Fresh session; select option 5 without transactions	Message indicates that no transactions are available.	**PASS**
TC15	PIN change and persistence	Change PIN from current 5678 to 1234, confirm, restart and log in with 1234	PIN change succeeds and new PIN works after restart.	**PASS**

7.2 Testing observations

The test report marks every listed case as PASS. The cases exercise normal operations and selected failure conditions, including invalid PINs, invalid withdrawal amounts, minimum-balance protection, daily withdrawal limits, UPI limits, fraud alerts, empty history, and PIN persistence.

7.3 Alignment note for the test evidence

The test report's TC15 example uses 5678 as the current PIN, while the source code uses 1234 as the default PIN when no valid pin.txt exists. The TC15 scenario is therefore consistent when the PIN has previously been changed to 5678, or when a matching saved PIN file is present. The test report describes the test setup; the source code determines the default behaviour.

The daily withdrawal counter and transaction history are session-based in the supplied code. Accordingly, the daily-limit test should be run in a fresh program session as stated in the test report.

8. RESULTS AND DISCUSSION

The submitted test report records PASS for all 15 test cases. The project demonstrates a complete menu-driven flow from authentication to transaction operations and exit. It also demonstrates how validation rules can prevent invalid or disallowed operations before account values are updated.

The transaction record structure makes it possible to show recent account activity, while the denomination calculation demonstrates nested-loop logic. The PIN file provides a simple way to retain a changed PIN across program restarts. The fraud-alert counter demonstrates a basic rule-based alert mechanism.

The result is an educational model of selected banking workflows, not a production banking application. Its value is in demonstrating programming fundamentals and translating simple transaction rules into code.

9. LIMITATIONS AND FUTURE ENHANCEMENTS

9.1 Limitations

• PIN is stored in a local plain-text file rather than encrypted storage.

• Transaction history is held in memory and is lost when the program ends.

• Daily withdrawal tracking resets when the program restarts.

• Card blocking is limited to the current program session and is not persistent.

• UPI ID validation checks only a basic string pattern and does not verify a real account.

• The simulator does not connect to a real bank, ATM, or UPI payment network.

9.2 Future enhancements

• Use encrypted PIN storage and stronger authentication controls.

• Store account and transaction data persistently in a database.

• Persist daily limits and account-blocking state across restarts.

• Add multiple user accounts and account-specific transaction histories.

• Add transaction timestamps, receipts, and improved reporting.

• Build a graphical user interface and stronger fraud-monitoring rules.

10. CONCLUSION

The ATM and UPI Transaction Simulator demonstrates how fundamental Python programming concepts can be applied to develop a simple banking-operations simulation. It provides practical experience in input validation, loops, functions, data structures, file handling, and exception handling. The project brings these concepts together in a menu-driven workflow covering authentication, balance enquiry, cash withdrawal, deposits, UPI transfers, transaction history, PIN management, and exit.

The project fulfils its educational purpose while clearly identifying areas that would need improvement before any real-world banking use. Persistent storage, stronger security, multiple accounts, and more advanced monitoring are suitable directions for future development.

APPENDIX A – COMPLETE PYTHON SOURCE CODE

The following listing reproduces the Python source code supplied for this report. It is included as a reference appendix.

```
"""
Project: ATM and UPI Transaction Simulator
Purpose: Simulate basic ATM and UPI banking operations.
"""

import os
from getpass import getpass


# ------------------------------------------------
# 1. INITIAL SETTINGS
# ------------------------------------------------

PIN_FILE = "pin.txt"

# Load the saved PIN or use the default PIN.
if os.path.exists(PIN_FILE):
    with open(PIN_FILE, "r") as file:
        correct_pin = file.read().strip()

    # Check whether the saved PIN is valid.
    if len(correct_pin) != 4 or not correct_pin.isdigit():
        correct_pin = "1234"
else:
    correct_pin = "1234"

balance = 10000.00

minimum_balance = 500.00
daily_withdrawal_limit = 20000.00
upi_limit = 5000.00
large_transfer_limit = 3000.00

daily_withdrawn = 0.00
large_transfer_streak = 0

denominations = (500, 200, 100)

# Each transaction is stored as a tuple inside this list.
transactions = []


# ------------------------------------------------
# 2. TRANSACTION RECORDING FUNCTION
# ------------------------------------------------

def record_transaction(transaction_type, amount):
    """Store transaction details in a tuple."""

    transaction_id = len(transactions) + 1

    transaction = (
```

```
        transaction_id,
        transaction_type,
        amount,
        balance
    )

    transactions.append(transaction)


# ------------------------------------------------
# 3. PIN VERIFICATION
# ------------------------------------------------

login_successful = False

print("\\n===================================")
print("       WELCOME TO ABC BANK")
print("===================================")

for attempt in range(1, 4):

    entered_pin = getpass("Enter your 4-digit PIN: ")

    if entered_pin == correct_pin:
        print("\\nLogin successful!")
        login_successful = True
        break

    else:
        attempts_left = 3 - attempt
        print("Incorrect PIN.")

        if attempts_left > 0:
            print("Attempts remaining:", attempts_left)

else:
    print("\\nYour card has been blocked.")
    print("You have entered the wrong PIN 3 times.")

# The for-else block runs only when the loop finishes
# without executing break.


# ------------------------------------------------
# 4. ATM MENU
# ------------------------------------------------

if login_successful:

    while True:

        print("\\n===================================")
```

```
print("             ATM MENU")
print("==================================")

print("1. Balance Enquiry")
print("2. Cash Withdrawal")
print("3. Deposit")
print("4. UPI Transfer")
print("5. Mini-Statement")
print("6. Change PIN")
print("7. Exit")

print("==================================")

choice = input("Enter your choice: ").strip()

# Check for invalid menu choices.
if choice not in ("1", "2", "3", "4", "5", "6", "7"):
    print("Invalid menu choice. Try again.")
    large_transfer_streak = 0
    continue

# Any non-UPI menu option breaks the
# consecutive large UPI transfer sequence.
if choice != "4":
    large_transfer_streak = 0


# ----------------------------------------
# OPTION 1: BALANCE ENQUIRY
# ----------------------------------------

if choice == "1":

    print("\\nCurrent balance: Rs.", balance)


# ----------------------------------------
# OPTION 2: CASH WITHDRAWAL
# ----------------------------------------

elif choice == "2":

    try:
        amount = float(input("Enter withdrawal amount: Rs. "))

    except ValueError:
        print("Invalid input. Enter a numeric amount.")
        continue

    if amount <= 0:
        print("Amount must be greater than zero.")
        continue
```

```
if amount != int(amount) or int(amount) % 100 != 0:
    print("Invalid amount. Enter multiples of Rs. 100.")
    continue

amount = int(amount)

# Check daily withdrawal limit.
if daily_withdrawn + amount > daily_withdrawal_limit:
    print("Daily withdrawal limit exceeded.")
    continue

# Check minimum balance requirement.
if balance - amount < minimum_balance:
    print("Insufficient balance.")
    print("You must maintain Rs. 500 minimum balance.")
    continue

# Cash dispenser algorithm.
remaining = amount
cash_notes = []

for note in denominations:

    count = 0

    while remaining >= note:
        remaining = remaining - note
        count = count + 1

    if count > 0:
        cash_notes.append((note, count))

    if remaining == 0:
        break

# Check whether the amount can be dispensed.
if remaining != 0:
    print("Sorry, this amount cannot be dispensed.")
    continue

# Update balance and daily withdrawal.
balance = balance - amount
daily_withdrawn = daily_withdrawn + amount

record_transaction("Cash Withdrawal", amount)

print("\\nWithdrawal successful!")
print("Amount withdrawn: Rs.", amount)

print("\\nCash denominations:")
```

```
    for note, count in cash_notes:
        print("Rs.", note, "x", count)

    print("Remaining balance: Rs.", balance)


# ----------------------------------------
# OPTION 3: DEPOSIT
# ----------------------------------------

elif choice == "3":

    try:
        amount = float(input("Enter deposit amount: Rs. "))

    except ValueError:
        print("Invalid input. Enter a numeric amount.")
        continue

    if amount <= 0:
        print("Deposit amount must be greater than zero.")
        continue

    balance = balance + amount

    record_transaction("Deposit", amount)

    print("\\nDeposit successful!")
    print("Amount deposited: Rs.", amount)
    print("Updated balance: Rs.", balance)


# ----------------------------------------
# OPTION 4: UPI TRANSFER
# ----------------------------------------

elif choice == "4":

    upi_id = input("Enter receiver's UPI ID: ").strip()

    # Basic UPI ID format validation.
    if (
        "@" not in upi_id
        or upi_id.startswith("@")
        or upi_id.endswith("@")
        or " " in upi_id
    ):
        print("Invalid UPI ID.")
        large_transfer_streak = 0
        continue

    try:
```

```
            amount = float(input("Enter transfer amount: Rs. "))

        except ValueError:
            print("Invalid input. Enter a numeric amount.")
            large_transfer_streak = 0
            continue

        if amount <= 0:
            print("Transfer amount must be greater than zero.")
            large_transfer_streak = 0
            continue

        if amount > upi_limit:
            print("UPI limit exceeded.")
            print("Maximum transfer per transaction is Rs. 5000.")
            large_transfer_streak = 0
            continue

        if balance - amount < minimum_balance:
            print("Insufficient balance.")
            print("Minimum balance of Rs. 500 must be maintained.")
            large_transfer_streak = 0
            continue

        # Complete the UPI transfer.
        balance = balance - amount

        record_transaction("UPI Transfer", amount)

        print("\\nUPI transfer successful!")
        print("Receiver:", upi_id)
        print("Amount transferred: Rs.", amount)
        print("Remaining balance: Rs.", balance)

        # Check for consecutive large transfers.
        if amount >= large_transfer_limit:

            large_transfer_streak = large_transfer_streak + 1

            if large_transfer_streak == 3:
                print("\\nFRAUD ALERT!")
                print("Three consecutive large UPI transfers detected.")
                large_transfer_streak = 0

        else:
            large_transfer_streak = 0


    # ----------------------------------------
    # OPTION 5: MINI-STATEMENT
    # ----------------------------------------
```

```
elif choice == "5":

    print("\\n========== MINI-STATEMENT ==========")

    if len(transactions) == 0:
        print("No transactions available.")

    else:
        # Display the last five transactions.
        recent_transactions = transactions[-5:]

        print("ID\\tType\\t\\tAmount\\t\\tBalance")
        print("-------------------------------------------")

        for transaction in recent_transactions:

            transaction_id, transaction_type, amount, current_balance =
            transaction

            print(
                transaction_id,
                "\\t",
                transaction_type,
                "\\t",
                "Rs.", amount,
                "\\t",
                "Rs.", current_balance
            )

    print("-------------------------------------------")


# ----------------------------------------
# OPTION 6: CHANGE PIN
# ----------------------------------------

elif choice == "6":

    current_pin = getpass("Enter current PIN: ")

    if current_pin != correct_pin:
        print("Incorrect current PIN.")
        continue

    new_pin = getpass("Enter new 4-digit PIN: ")

    if len(new_pin) != 4 or not new_pin.isdigit():
        print("PIN must contain exactly 4 digits.")
        continue

    if new_pin == correct_pin:
        print("New PIN must be different from the old PIN.")
```

```
            continue

        confirm_pin = getpass("Confirm new PIN: ")

        if new_pin != confirm_pin:
            print("PIN confirmation does not match.")
            continue

        # Update and save the new PIN.
        correct_pin = new_pin

        with open(PIN_FILE, "w") as file:
            file.write(correct_pin)

        record_transaction("PIN Change", 0)

        print("\\nPIN changed successfully!")
        print("Your new PIN has been saved.")


    # ----------------------------------------
    # OPTION 7: EXIT
    # ----------------------------------------

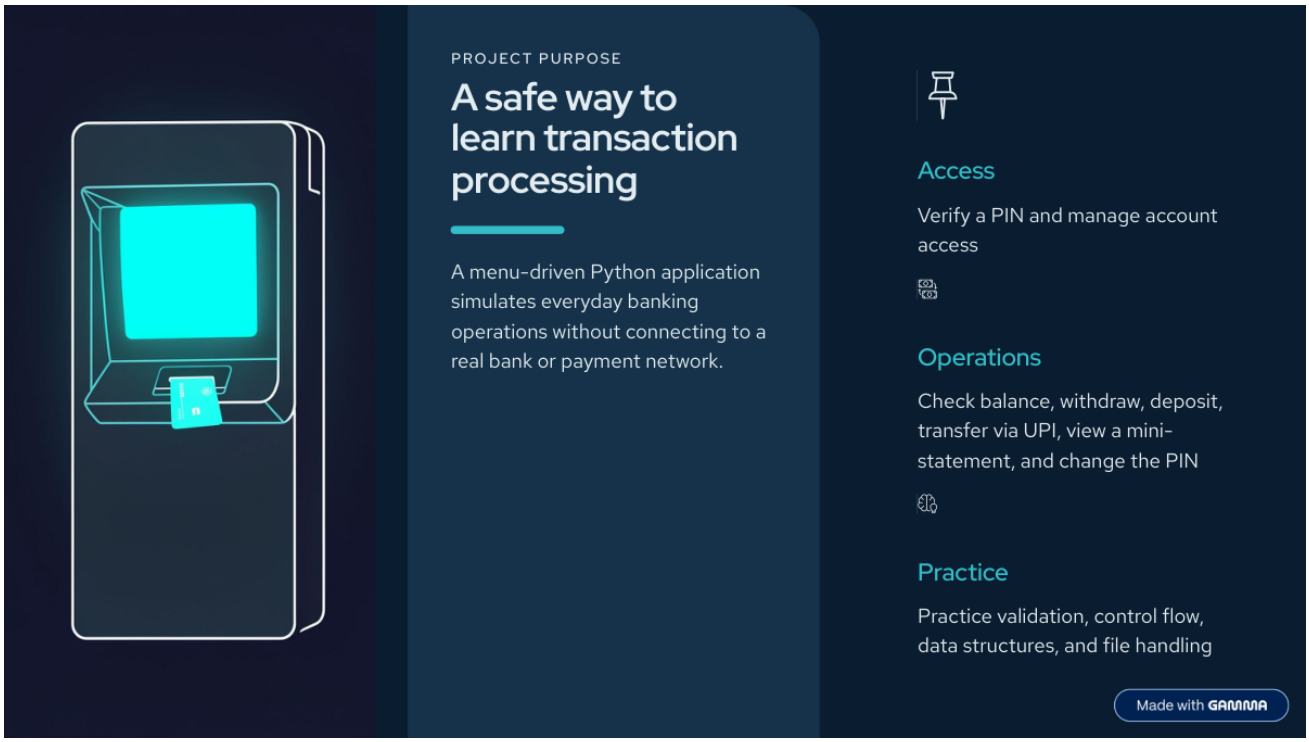
    elif choice == "7":

        print("\\nThank you for using ABC Bank!")
        print("Please visit again.")
        break
```

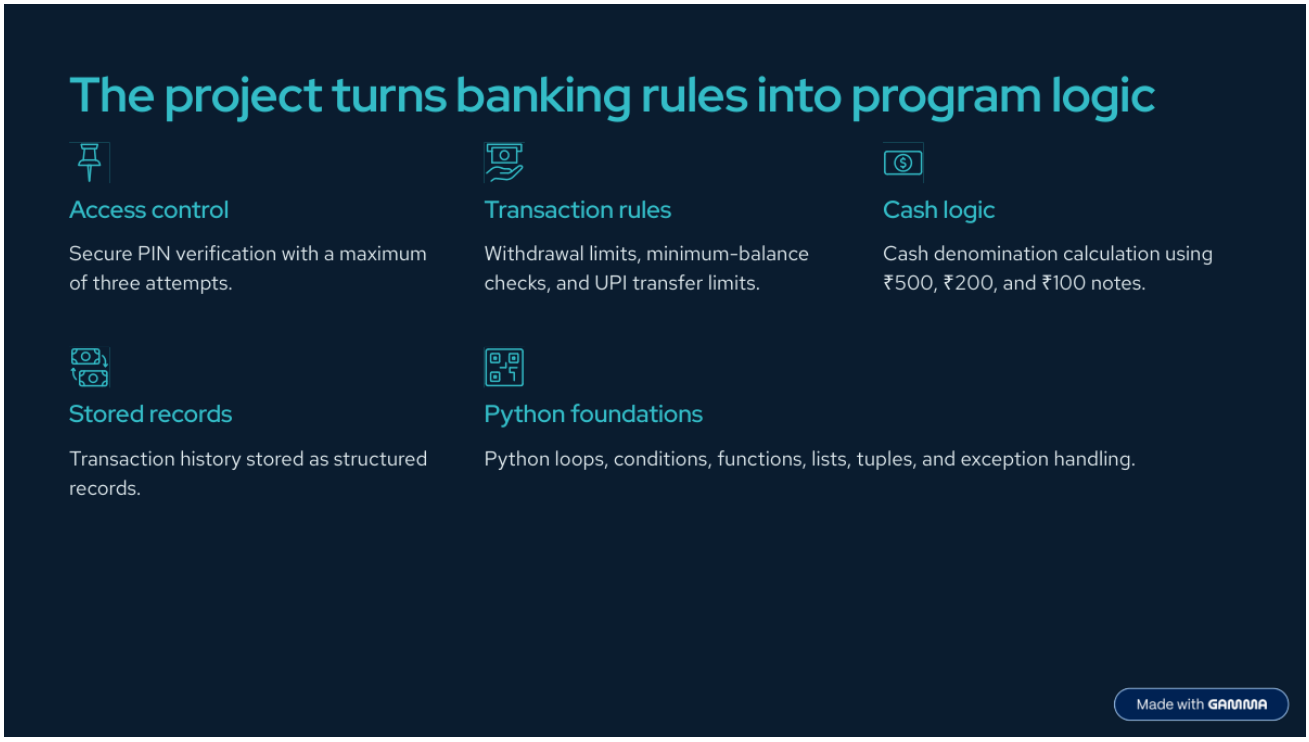
APPENDIX B – PRESENTATION VISUALS

Selected pages from the supplied presentation are reproduced below as visual references. They retain the original presentation content and design.

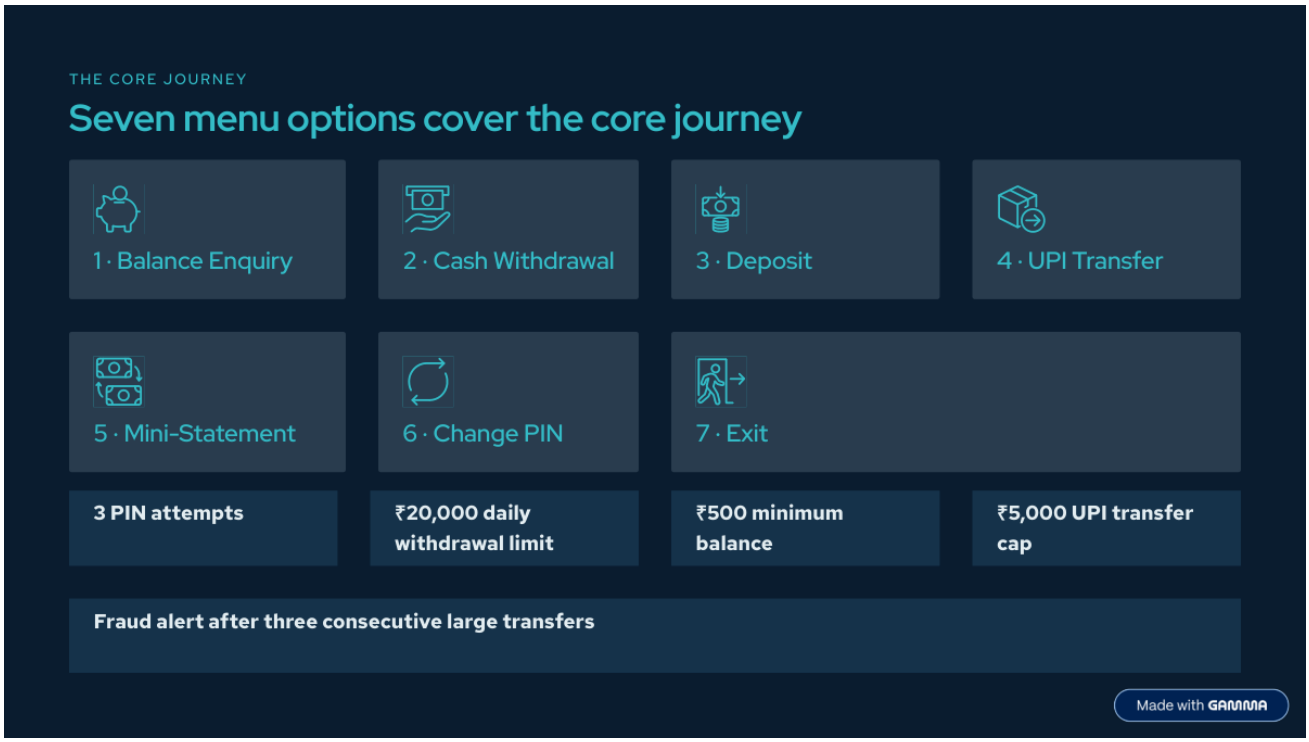
Presentation page 2



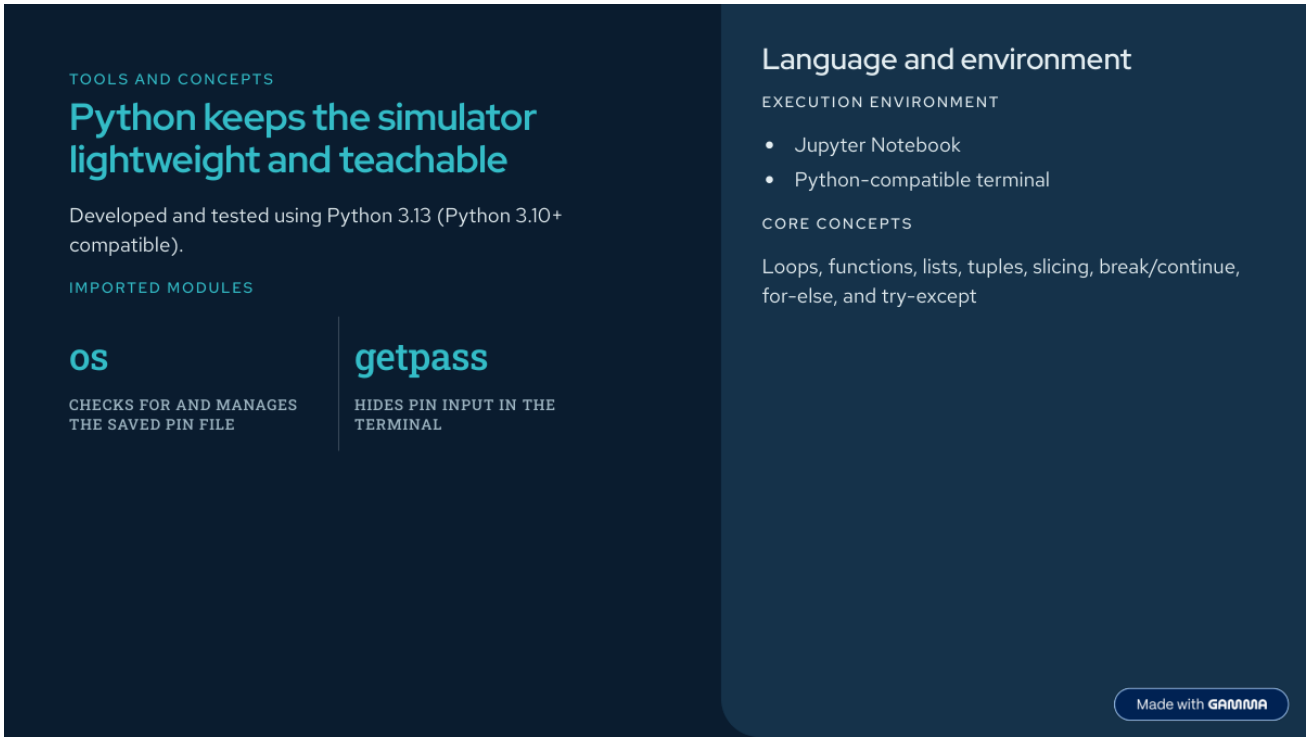
Presentation page 3



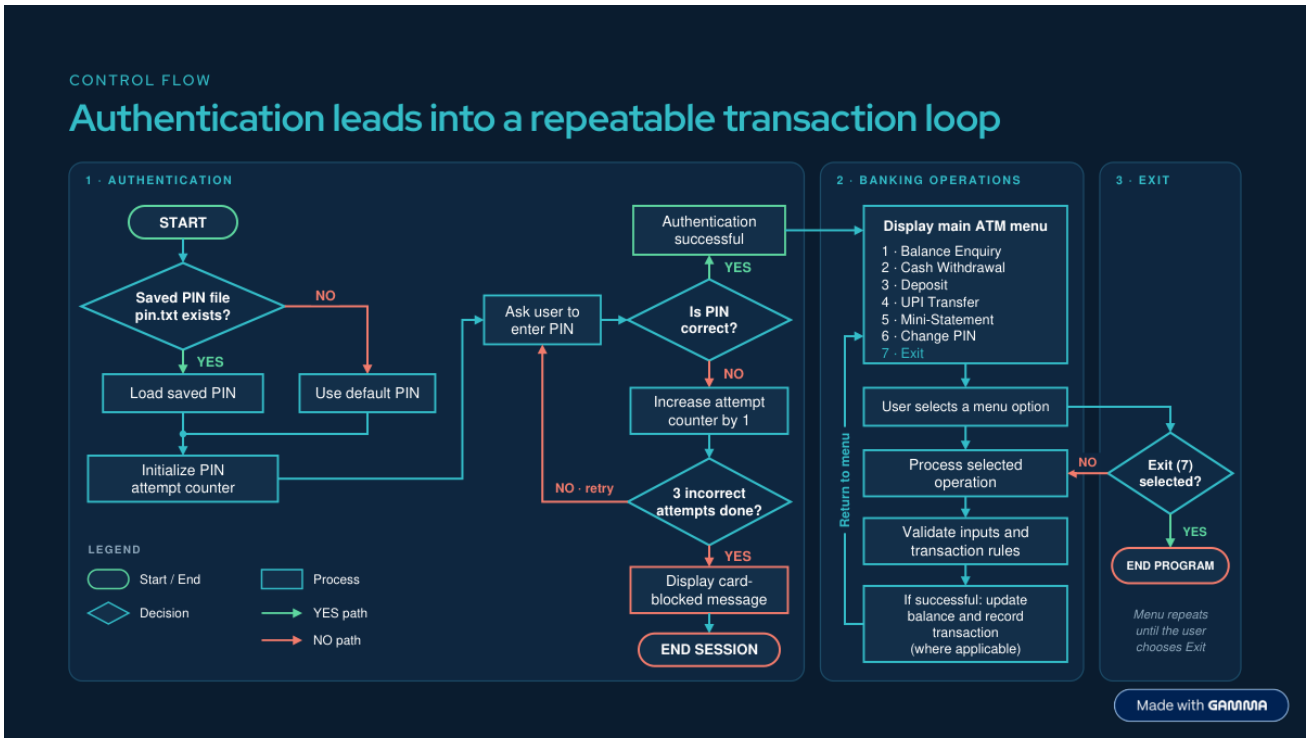
Presentation page 4



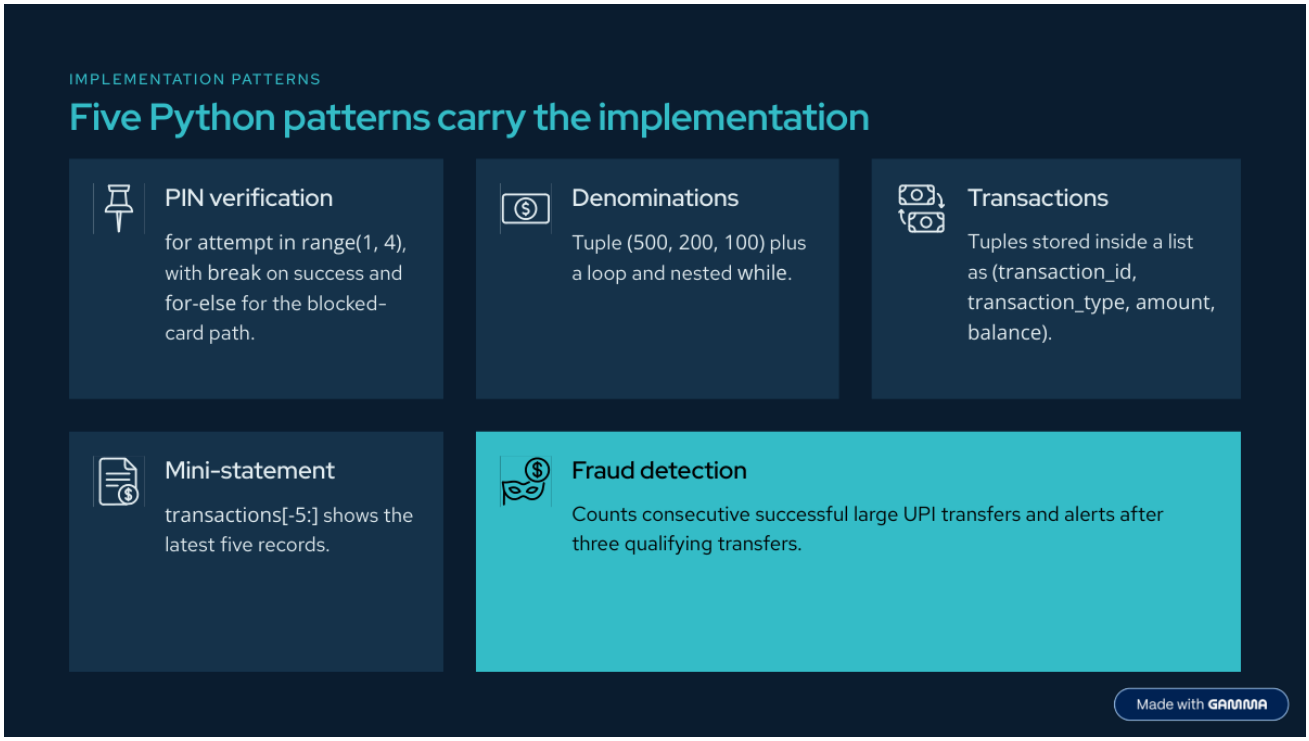
Presentation page 5



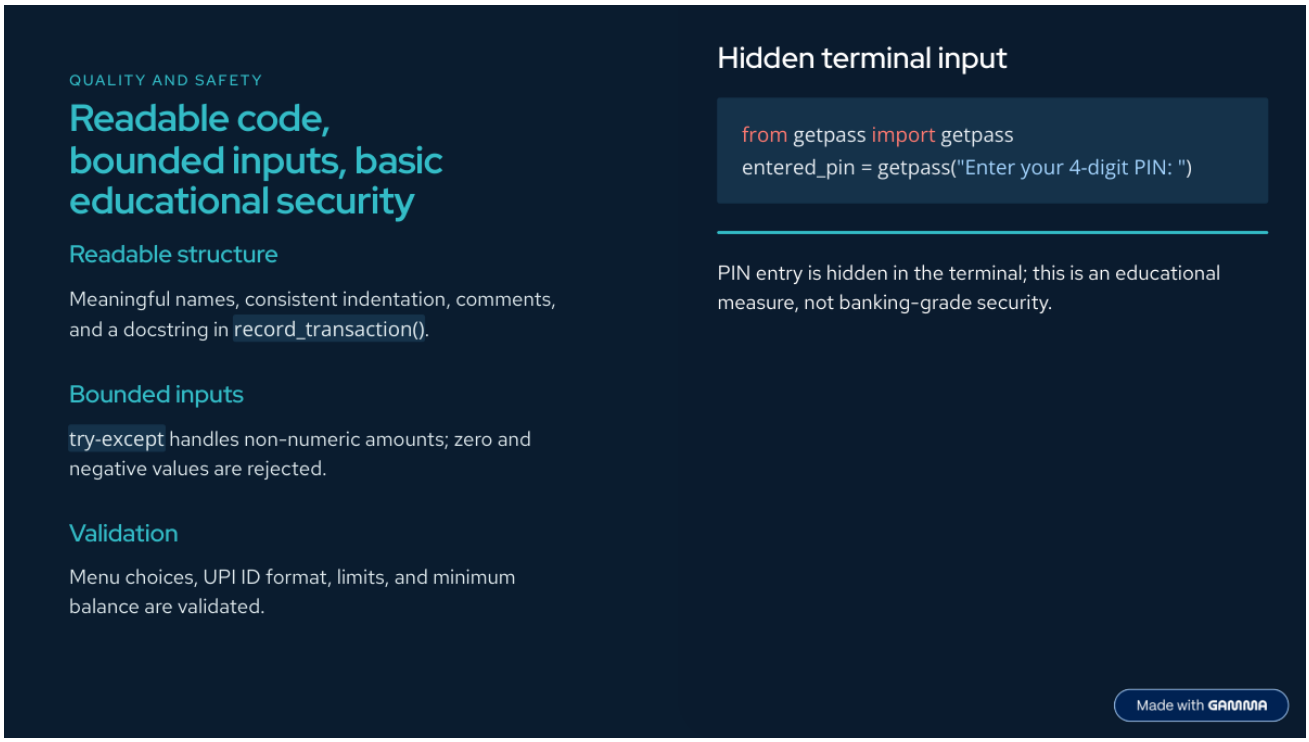
Presentation page 6



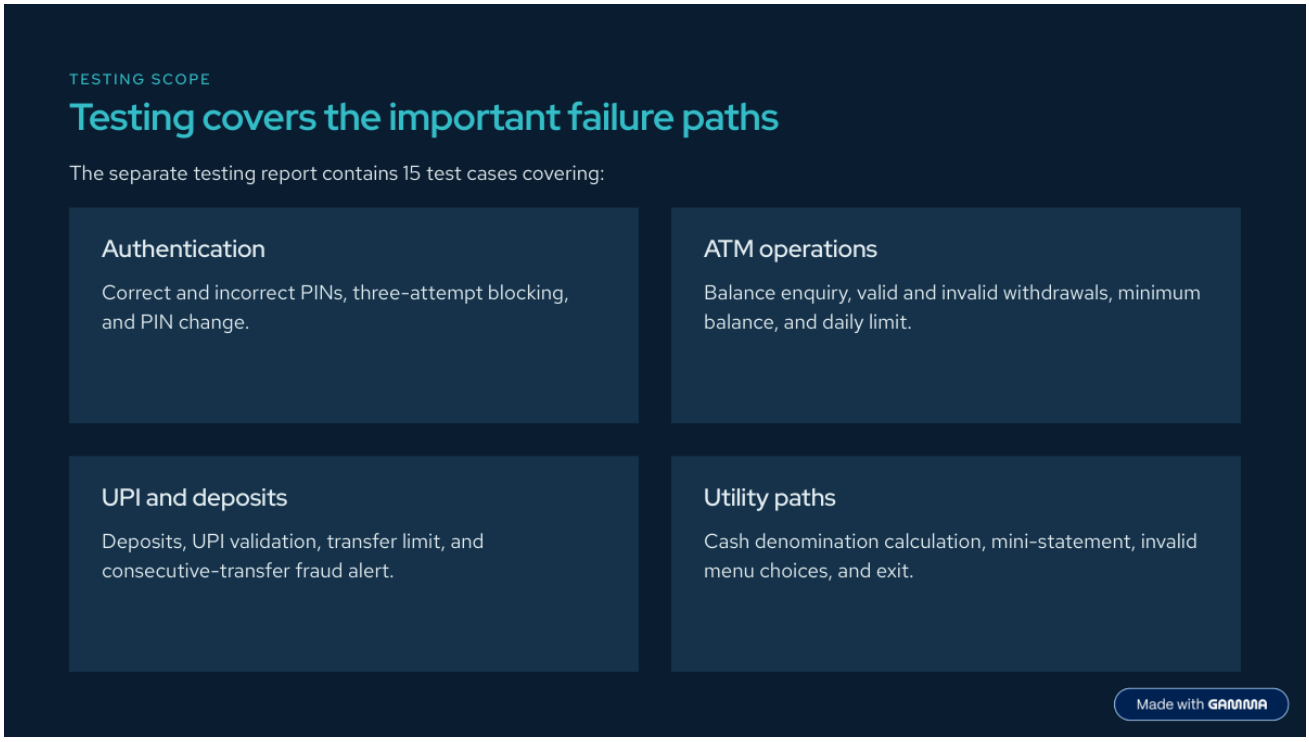
Presentation page 7



Presentation page 8



Presentation page 9



Presentation page 10

TAKEAWAY

The simulator is intentionally a learning system

Limitations

PIN stored locally in a plain-text file. Session-only transaction history, daily withdrawal tracking resets, card blocking is not persistent, and basic UPI ID format validation.

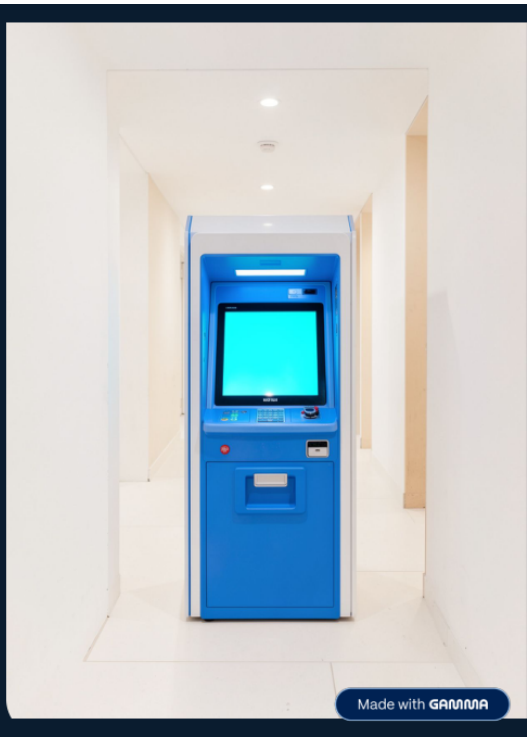
Future

Database persistence, encrypted PIN storage, permanent history, GUI, multiple accounts, timestamps, and stronger monitoring.

Conclusion

The project connects Python fundamentals to a realistic transaction workflow.

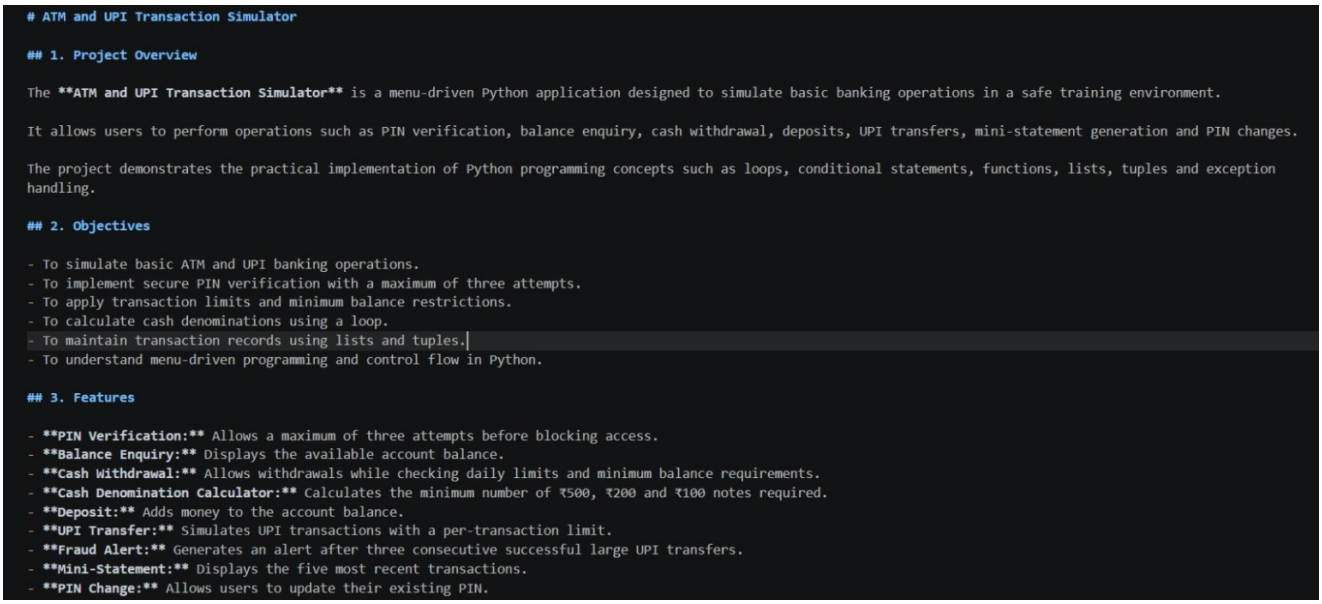
THANK YOU



10

Made with GAMMA

Project overview and objectives screenshot



```
# ATM and UPI Transaction Simulator

## 1. Project Overview

The **ATM and UPI Transaction Simulator** is a menu-driven Python application designed to simulate basic banking operations in a safe training environment.

It allows users to perform operations such as PIN verification, balance enquiry, cash withdrawal, deposits, UPI transfers, mini-statement generation and PIN changes.

The project demonstrates the practical implementation of Python programming concepts such as loops, conditional statements, functions, lists, tuples and exception handling.

## 2. Objectives

- To simulate basic ATM and UPI banking operations.
- To implement secure PIN verification with a maximum of three attempts.
- To apply transaction limits and minimum balance restrictions.
- To calculate cash denominations using a loop.
- To maintain transaction records using lists and tuples.
- To understand menu-driven programming and control flow in Python.

## 3. Features

- **PIN Verification:** Allows a maximum of three attempts before blocking access.
- **Balance Enquiry:** Displays the available account balance.
- **Cash Withdrawal:** Allows withdrawals while checking daily limits and minimum balance requirements.
- **Cash Denomination Calculator:** Calculates the minimum number of ₹500, ₹200 and ₹100 notes required.
- **Deposit:** Adds money to the account balance.
- **UPI Transfer:** Simulates UPI transactions with a per-transaction limit.
- **Fraud Alert:** Generates an alert after three consecutive successful large UPI transfers.
- **Mini-Statement:** Displays the five most recent transactions.
- **PIN Change:** Allows users to update their existing PIN.
```

Project conclusion screenshot

